

TalktoMe — Project Report

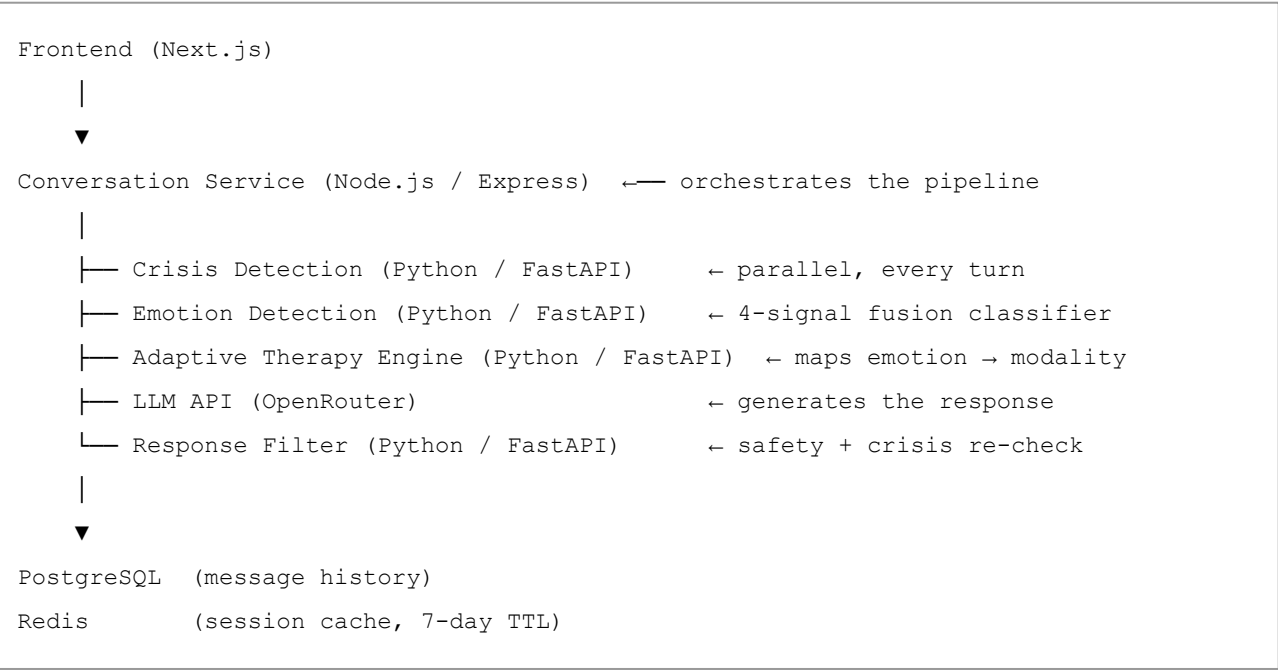
What It Is

TalktoMe is a privacy-first AI emotional support companion. Users can share what's on their mind in a chat interface and receive responses that adapt in real time to their emotional state — switching between therapeutic approaches such as validation, CBT, mindfulness, and journaling depending on what the current turn signals.

It is not a therapy tool. It is a low-barrier, judgment-free space for people who need to process something but can't say it to another person.

Architecture

A microservices pipeline. Every message passes through a sequential chain of independent services:



Tech Stack

Layer	Technology
Web frontend	Next.js 16, React 19, TypeScript
Conversation service	Node.js 22, Express
ML services	Python 3.14, FastAPI, uvicorn
LLM	OpenRouter (claude-3.5-haiku by default)
Database	PostgreSQL 16 (via Docker)

Layer	Technology
Cache	Redis 7
Infrastructure	Docker Compose

Design Patterns Used

Observer (conversation service)

The `chatEmitter` is a singleton `AsyncEventEmitter`. After a reply is generated, it emits `reply` and `crisis` events.

Two registered observers handle side effects independently:

- **`persistenceObserver`** — saves user + assistant messages to Postgres
- **`logObserver`** — logs crisis flags and safety fallbacks

This decouples the pipeline logic from persistence and logging. `chat.js` never calls `saveMessage` directly.

Composite (filter service)

`CompositeSafetyCheck` holds a list of `SafetyCheck` instances and runs them in sequence. Adding a new check requires no change to existing code — just add it to the list in `app.py`. Currently composed of `PatternSafetyCheck` + `CrisisRecheckCheck`.

Template Method (ATE modalities)

`Modality` is an abstract base class with:

- `name` — abstract property
- `guidance()` — abstract, returns the core instruction
- `avoid()` — concrete with a default of `[]`, overridden per modality
- `full_guidance()` — concrete, combines `guidance()` + `avoid()` into the final injected string

Each of the six modality classes (CBT, Validation, Journaling, Mindfulness, SFBT, Narrative) implements the abstract methods. The engine calls only `full_guidance()` — it doesn't know which concrete modality it's working with.

Strategy (ATE engine)

`engine.py` maintains a map of `emotion_label` → `Modality` instance. At runtime it selects the appropriate strategy (modality) based on the detected emotion and confidence score, with a fallback to `Validation` when confidence is below the threshold (0.30).

Principles Applied

Single Responsibility — each service does one thing. The conversation service orchestrates; it does not classify, generate, or filter. Each Python service has one job.

Open/Closed — the filter's Composite structure and the modality Template Method both allow extension without modification. New safety checks and new therapy modalities can be added by writing a new class, not editing existing ones.

Fail-safe defaults — Redis cache failures are caught silently; the service falls back to Postgres. LLM semantic signal failures fall back to lexical + explicit only. Crisis detection defaults to showing resources on ambiguity.

Separation of concerns — context window assembly, session caching, prompt building, and HTTP routing are each in their own module in the conversation service.

Privacy by structure — no message content is written to logs anywhere. The observer logs only session ID and event type. Redis entries expire automatically.

Current Status

Component	Status
Chat pipeline (end-to-end)	Working
Emotion detection (4 signals)	Working
ATE (6 modalities)	Working
Crisis detection	Working (keyword-based)
Response filter	Working
Redis session cache	Working (lazy connect, graceful fallback)
Frontend (web, with sidebar + history)	Working
Long-term memory (theme vectors)	Not built — table exists, nothing writes to it
Mobile client	Not started — default RN scaffold only

TalktoMe is an AI companion — not a therapist. Crisis resources: iCall 9152987821 · Vandrevalla Foundation 1860-2662-345.